COMBINING MERTON (1976) AND HESTON (1993)

Reading Time	75 minutes
Prior Knowledge	Merton model, Heston model, Fourier transform, Characteristic function, Lewis, FFT
Keywords	Bates, Lewis, FFT

In this lesson we will introduce a new model: Bates (1996). The main idea of Bates is simple, combining desirable features such as stochastic volatility (Heston, 1993) and jump-diffusion (Merton, 1976) into one model.

1. Bates (1996) model

Bates (1996) is a stochastic volatility jump-diffusion model with the following risk-neutral dynamics:

$$ \begin{equation} dS_t = (r_t - r_J) S_t dt + \sqrt{\nu_t} S_t dZ_t^1 + J_t S_t dN_t \end{equation} $$

$$ \begin{equation} d\nu_t = \kappa_\nu (\theta_\nu - \nu_t)dt + \sigma_\nu \sqrt{\nu_t}dZ_t^2 \end{equation} $$

where,

- r_t is the short risk-free rate at date t
- $r_J \equiv \lambda (e{\mu_J `\delta^2/2}-1)$ is the drift correction for jump
- ν_t is the variance at date t
- κ_ν is the speed of adjustment of ν_t to θ_ν, the long-term variance
- σ_ν volatility of variance
- Z_t^n are standard Brownian motions

- N_t is a Poisson process with intensity λ
- J_t is a jump at date t with $log(1+J_t)$ following a normal distribution $N(log(1+\mu_J)-\frac{\delta^2}{2}, \delta^2)$

1.1. Unveiling Bates (1996) characteristic function

As we already know, the most important ingredient to price and calibrate a model such Bates using Fourier methods is its **characteristic function**. So, what's Bates CF?

Bates (1996) combines two basic models: Heston (1993) and Merton (1976). Thus, it makes sense that its characteristic function is a combination of these two, which we already know.

To get Bates' CF, we will take the characteristic function of Heston (1993) as is, but we will need to adjust that of Merton (1976) because we just need the part of the jump (not the diffusive part, which is already given by Heston). Thus, the 'jump-part' characteristic function of Merton (1976), $ \varphi^{M76J}_0 (u, T)$, becomes:

$$ \varphi^{M76J}_0 (u, T) = e^{\left(\left(i u \omega + \lambda (e^{i u \mu_j - u^2 \delta^2/2}-1) \right) T \right)} $$

where,

$$ \omega = - \lambda \left(e^{\mu_j + \delta^2/2}-1 \right) $$

Once we have adjusted the Merton (1976) characteristic function, we can recognize that jumps occur independently of the drift process. That is, **correlation between the trend component of the asset price and jump component is zero**

Hence, we can express the characteristic function of Bates (1996) simply as the product of two characteristic functions:

$$ \varphi^{B96}_0 (u, T) = \varphi^{H93}_0 \varphi^{M76J}_0 (u, T) $$

where φ^{H93}_0 stands for the characteristic function of Heston (1993):

$$ \varphi^{H93} (u, T) = e^{H_1(u, T)+H_2(u,T)\nu_0} $$

$$ H_1 (u, T) \equiv r_0 uiT + \frac{c_1}{\sigma_\nu^2}\Biggl\{ (\kappa_\nu - \rho \sigma_\nu ui+c_2) T - 2 log \left[\frac{1-c_3e^{c_2T}}{1-c_3} \right] \Biggl\} $$

$$ H_2 (u, T) \equiv \frac{\kappa_\nu - \rho \sigma_\nu ui + c_2}{\sigma_\nu^2} \left[\frac{1-e^{c_2T}}{1-c_3e^{c_2T}} \right] $$

$$ c_1 \equiv \kappa_\nu \theta_\nu $$

$$ c_2 \equiv - \sqrt{(\rho \sigma_\nu ui - \kappa_\nu)^2 - \sigma_\nu^2(-ui-u^2) } $$

$$ c_3 \equiv \frac{\kappa_\nu - \rho \sigma_\nu ui + c_2}{\kappa_\nu - \rho \sigma_\nu ui - c_2} $$

2. Bates (1996) in practice

Before jumping to the calibration stage in Bates, let's do a simple pricing exercise using Fourier methods under Lewis (2001) and Carr-Madan (1999) approaches.

For both of these, the first crucial ingredient to code is the characteristic function:

2.1. CF in Bates (1996)

The characteristic function of Bates (1996) was given by:

$$ \varphi^{B96}_0 (u, T) = \varphi^{H93}_0 \varphi^{M76J}_0 (u, T) $$

which, as we already know, is essentially the product of two characteristic functions: Heston (1993) and a modified version of Merton (1976):

In [1]:
```
import numpy as np
from scipy.integrate import quad
```

In [2]:
```
# Heston (1993) characteristic function

def H93_char_func(u, T, r, kappa_v, theta_v, sigma_v, rho, v0):
    """Valuation of European call option in H93 model via Lewis (2001)
    Fourier-based approach: characteristic function.
    Parameter definitions see function BCC_call_value."""
    c1 = kappa_v * theta_v
    c2 = -np.sqrt(
        (rho * sigma_v * u * 1j - kappa_v) ** 2 - sigma_v**2 * (-u * 1j -
    )
    c3 = (kappa_v - rho * sigma_v * u * 1j + c2) / (
```

```
        kappa_v - rho * sigma_v * u * 1j - c2
    )
    H1 = r * u * 1j * T + (c1 / sigma_v**2) * (
        (kappa_v - rho * sigma_v * u * 1j + c2) * T
        - 2 * np.log((1 - c3 * np.exp(c2 * T)) / (1 - c3))
    )
    H2 = (
        (kappa_v - rho * sigma_v * u * 1j + c2)
        / sigma_v**2
        * ((1 - np.exp(c2 * T)) / (1 - c3 * np.exp(c2 * T)))
    )
    char_func_value = np.exp(H1 + H2 * v0)
    return char_func_value
```

In [3]:
```
# Merton (1976) modified CF

def M76J_char_func(u, T, lamb, mu, delta):
    """
    Adjusted Characteristic function for Merton '76 model: Only jump comp
    """

    omega = -lamb * (np.exp(mu + 0.5 * delta**2) - 1)
    char_func_value = np.exp(
        (1j * u * omega + lamb * (np.exp(1j * u * mu - u**2 * delta**2 *
        * T
    )
    return char_func_value
```

In [4]:
```
# Bates (1996) CF combining H93 and M76J

def B96_char_func(u, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu,
    """
    Bates (1996) characteristic function
    """
    H93 = H93_char_func(u, T, r, kappa_v, theta_v, sigma_v, rho, v0)
    M76J = M76J_char_func(u, T, lamb, mu, delta)
    return H93 * M76J
```

2.2. Lewis (2001) approach

For the Lewis approach, we need to further calculate the value of the integral in Lewis:

$$ C_0 = S_0 - \frac{\sqrt{S_0 K} e^{-rT}}{\pi} \int_{0}^{\infty} \mathbf{Re}[e^{izk} \varphi^{B96}(z-i/2)] \frac{dz}{z^2+1/4} $$

In [5]:
```
def B96_int_func(u, S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb
    """
    Lewis (2001) integral value for Bates (1996) characteristic function
    """
    char_func_value = B96_char_func(
        u - 1j * 0.5, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu,
    )
    int_func_value = (
        1 / (u**2 + 0.25) * (np.exp(1j * u * np.log(S0 / K)) * char_func_
```

```
    )
    return int_func_value
```

which, in turn, leads to the function for the value of the Call option under Lewis (2001):

In [6]:
```
def B96_call_value(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb,
    """
    Valuation of European call option in B96 Model via Lewis (2001)
    Parameters:
    ==========
    S0: float
        initial stock/index level
    K: float
        strike price
    T: float
        time-to-maturity (for t=0)
    r: float
        constant risk-free short rate
    kappa_v: float
        mean-reversion factor
    theta_v: float
        long-run mean of variance
    sigma_v: float
        volatility of variance
    rho: float
        correlation between variance and stock/index level
    v0: float
        initial level of variance
    lamb: float
        jump intensity
    mu: float
        expected jump size
    delta: float
        standard deviation of jump
    ==========
    """
    int_value = quad(
        lambda u: B96_int_func(
            u, S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu,
        ),
        0,
        np.inf,
        limit=250,
    )[0]
    call_value = max(0, S0 - np.exp(-r * T) * np.sqrt(S0 * K) / np.pi * i
    return call_value
```

2.3. Carr-Madan (1999) approach

As an alternative to Lewis (2001), we could also implement the FFT algorithm. Essentially, we can apply FFT to the integral in the call option price derived by Carr and Madan (1999)

$$ C_0 = \frac{e^{-\alpha \kappa}}{\pi} \int_{0}^{\infty} e^{-i\nu \kappa} \frac{e^{-rT}

\varphi^{B96} (\nu - (\alpha + 1)i, T)}{\alpha^2 + \alpha - \nu^2 + i(2\alpha + 1)\nu} d\nu $$

Here we are going to use the same numerical routine we implemented in the past; but note that this is essentially to ensure a well-behaved model.

As was the case with the Lewis (2001) approach, we basically have to adapt the characteristic function we are considering to be the Bates (1996) one.

In [7]:
```
def B96_call_FFT(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, m
    """
    Call option price in Bates (1996) under FFT
    """

    k = np.log(K / S0)
    g = 1  # Factor to increase accuracy
    N = g * 4096
    eps = (g * 150) ** -1
    eta = 2 * np.pi / (N * eps)
    b = 0.5 * N * eps - k
    u = np.arange(1, N + 1, 1)
    vo = eta * (u - 1)

    # Modifications to ensure integrability
    if S0 >= 0.95 * K:  # ITM Case
        alpha = 1.5
        v = vo - (alpha + 1) * 1j
        modcharFunc = np.exp(-r * T) * (
            B96_char_func(v, T, r, kappa_v, theta_v, sigma_v, rho, v0, la
            / (alpha**2 + alpha - vo**2 + 1j * (2 * alpha + 1) * vo)
        )

    else:
        alpha = 1.1
        v = (vo - 1j * alpha) - 1j
        modcharFunc1 = np.exp(-r * T) * (
            1 / (1 + 1j * (vo - 1j * alpha))
            - np.exp(r * T) / (1j * (vo - 1j * alpha))
            - B96_char_func(
                v, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu, de
            )
            / ((vo - 1j * alpha) ** 2 - 1j * (vo - 1j * alpha))
        )

        v = (vo + 1j * alpha) - 1j

        modcharFunc2 = np.exp(-r * T) * (
            1 / (1 + 1j * (vo + 1j * alpha))
            - np.exp(r * T) / (1j * (vo + 1j * alpha))
            - B96_char_func(
                v, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu, de
            )
            / ((vo + 1j * alpha) ** 2 - 1j * (vo + 1j * alpha))
        )

    # Numerical FFT Routine
    delt = np.zeros(N)
```

```
    delt[0] = 1
    j = np.arange(1, N + 1, 1)
    SimpsonW = (3 + (-1) ** j - delt) / 3
    if S0 >= 0.95 * K:
        FFTFunc = np.exp(1j * b * vo) * modcharFunc * eta * SimpsonW
        payoff = (np.fft.fft(FFTFunc)).real
        CallValueM = np.exp(-alpha * k) / np.pi * payoff
    else:
        FFTFunc = (
            np.exp(1j * b * vo) * (modcharFunc1 - modcharFunc2) * 0.5 * e
        )
        payoff = (np.fft.fft(FFTFunc)).real
        CallValueM = payoff / (np.sinh(alpha * k) * np.pi)

    pos = int((k + b) / eps)
    CallValue = CallValueM[pos] * S0

    return CallValue
```

2.4. Pricing example

We have built functions to price Call options under both Lewis and Carr-Madan/FFT approaches. Let's check how these pricing functions work with a simple example, assuming we had the parameter values of the Bates model:

In [8]:
```
# General Parameters
S0 = 36
K = 92
T = 165/365
r = 8.95/100

# Heston'93 Parameters
kappa_v = 0.04
theta_v = 0.43
sigma_v = 0.15
rho = -0.5
v0 = 0.26

# Merton'76 Parameters
lamb = 2.5
mu = -0.65
delta = 0.75
sigma = np.sqrt(v0)
```

In [9]:
```
print(
    "B96 Call option price via Lewis(2001): $%10.4f"
    % B96_call_value(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lam
)
```

```
B96 Call option price via Lewis(2001): $    2.0134
```

In [10]:
```
print(
    "B96 Call option price via Carr-Madan: $%10.4f"
    % B96_call_FFT(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb,
)
```

```
B96 Call option price via Carr-Madan: $    2.0134
```

3. Conclusion

In this lesson, we have applied both Fourier-based techniques learned in Module 1--Lewis (2001) and Carr and Madan (1999) FFT procedure--to the stochastic volatility jump-diffusion model of Bates (1996). As you can see, once we are familiar with Fourier pricing methods, it is just a matter of adapting the code to the characteristic function of the underlying asset process.

We still have not touched upon the most important issue, though: model calibration. That's what we will do in the next lesson.

References

- Bates, David S. "Jumps and Stochastic Volatility: Exchange Rate Processes Implicit in Deutsche Mark Options." *The Review of Financial Studies*, vol. 9, no. 1, 1996, pp. 69-107.
- Gatheral, Jim. *The Volatility Surface: A Practitioner's Guide*. John Wiley & Sons Inc., 2006.
- Hilpisch, Yves. *Derivatives Analytics with Python: Data Analysis, Models,Simulation, Calibration and Hedging*. John Wiley & Sons, 2015.
- Lewis, Alan L. "A Simple Option Formula for General Jump-Diffusion and Other Exponential Lévy Processes." 2001.
- Merton, Robert C. "Option pricing when underlying stock returns are discontinuous." Journal of financial economics 3.1-2 (1976): 125-144.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.